



SCHOOL OF
SCIENCE &
TECHNOLOGY



<https://www.techventurebootcamp.org/>

Welcome to the

GenAI Tutorial

Manoel Gadi

Dean & Professor
IE School of Science & Technology



IE Professor with Phd in Cryptocurrency, **Manoel Gadi** is a highly experienced analytics and business intelligence professional with a strong background in the financial sector. He holds a Master of Science in Computer Engineering from the University of São Paulo, Brazil, and an MBA from IE Business School in Madrid, and a PDH in cryptocurrency and AI from Universidad de Alcalá.

Throughout his career, Prof. Gadi has held key positions at prominent financial institutions, including Banco Santander and Citibank Brazil, where he specialized in leveraging data analytics for risk management, marketing, and sales optimization.

He possesses deep expertise in risk analytics, model management, Big Data, machine learning, and financial analytics, with a particular focus on credit risk modeling.

Currently, Prof. Gadi leads SunCaged Analytics Consulting Europe SL, providing specialized analytics consulting services.

Using ChatGPT

The screenshot shows the ChatGPT interface with three main sections: Examples, Capabilities, and Limitations. Each section contains three items. At the bottom, there is a text input field labeled 'Send a message...' and a disclaimer: 'ChatGPT Mar 23 Version. Free Research Preview. ChatGPT may produce inaccurate information about people, places, or facts.'

Examples	Capabilities	Limitations
"Explain quantum computing in simple terms" →	Remembers what user said earlier in the conversation	May occasionally generate incorrect information
"Got any creative ideas for a 10 year old's birthday?" →	Allows user to provide follow-up corrections	May occasionally produce harmful instructions or biased content
"How do I make an HTTP request in Javascript?" →	Trained to decline inappropriate requests	Limited knowledge of world and events after 2021

Also available to developers via OpenAI API

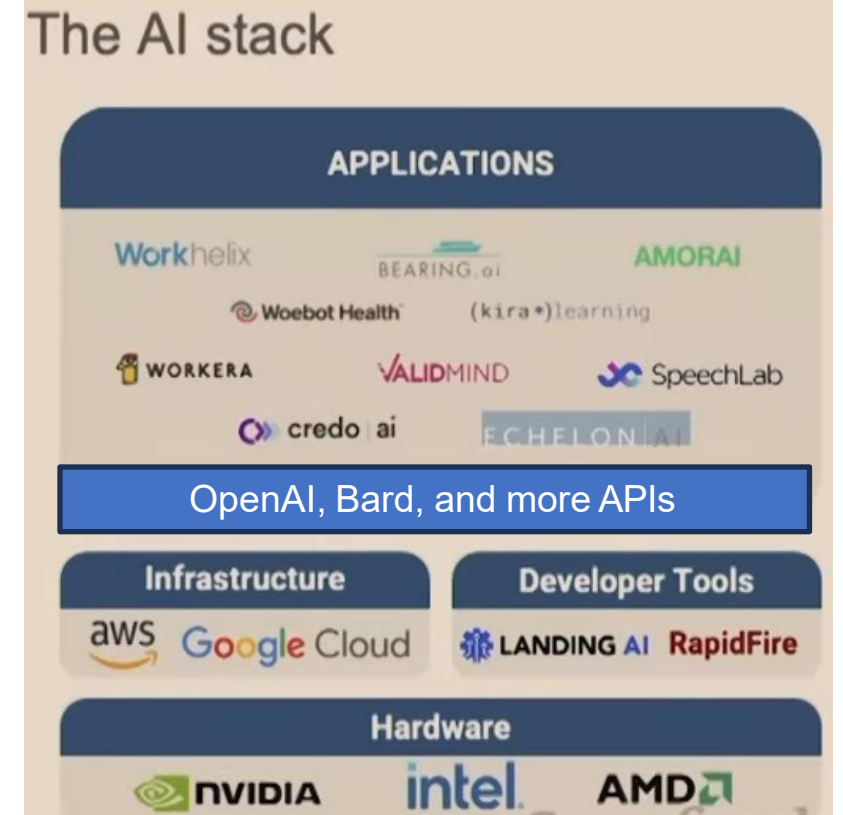
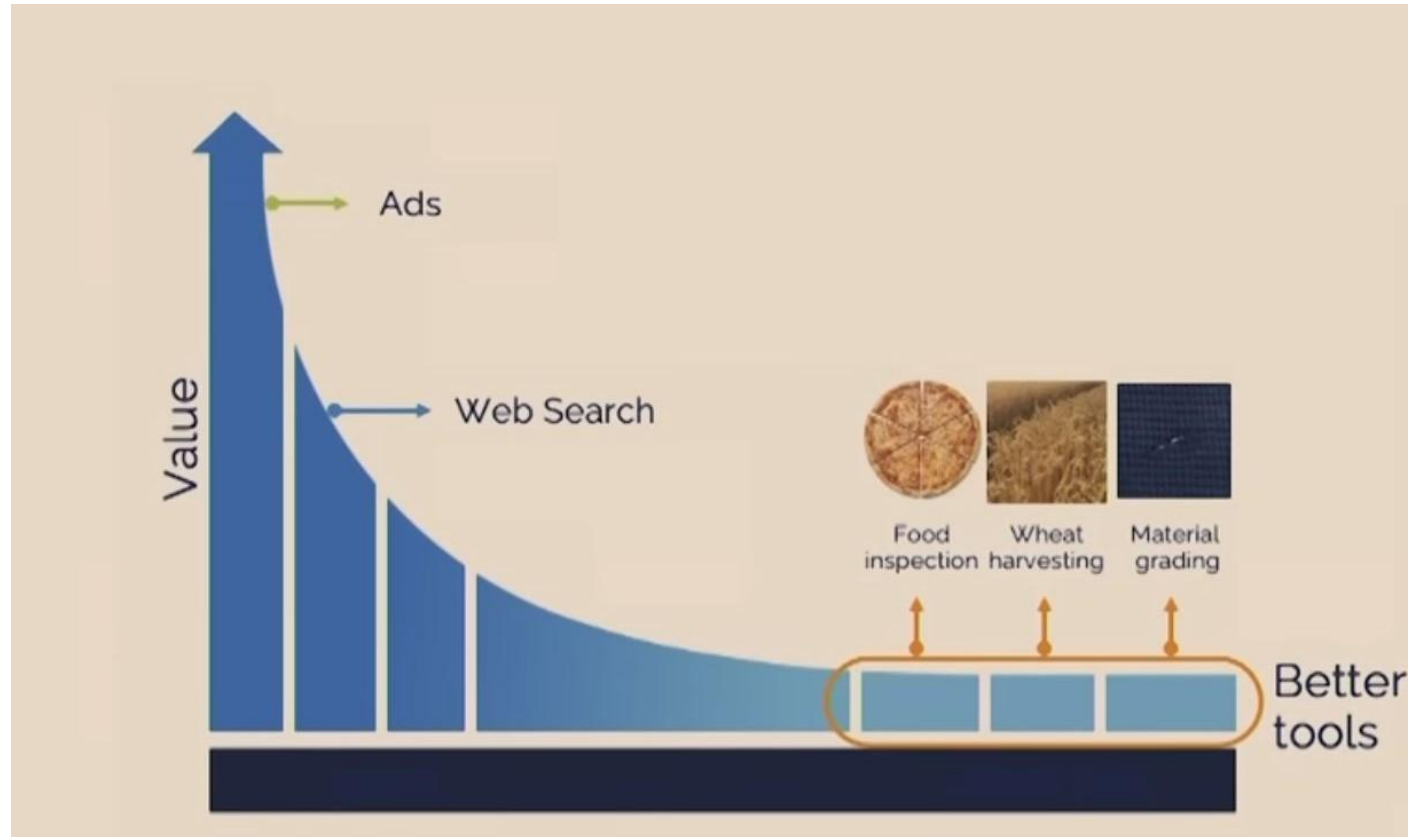
The screenshot shows a Jupyter Notebook interface with a code cell. The code uses the OpenAI API to get an answer to a question. Below the code, the output is displayed as a numbered list of five investment products.

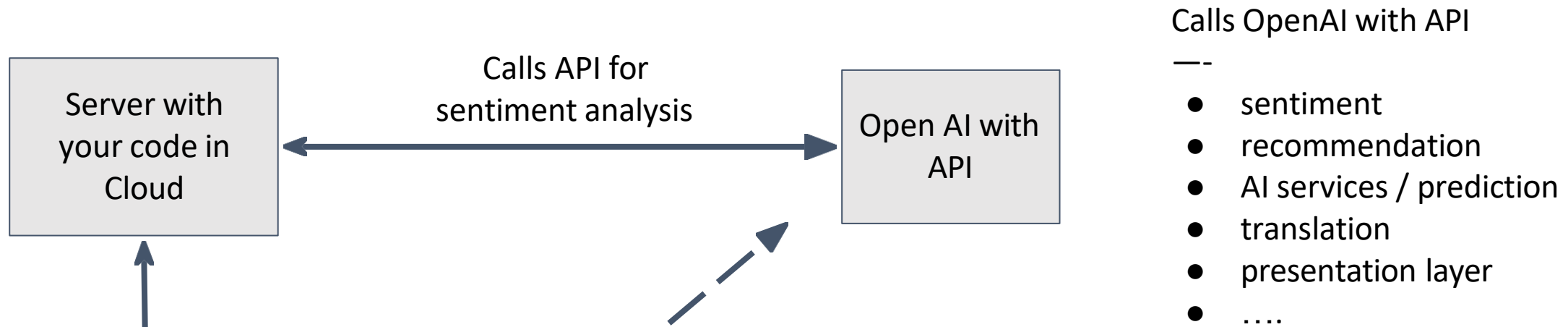
```
[5]: # Lets get an answer?  
  
q = "What are the most common investment products?. I want a short of 5 with 10 words"  
  
ans = openai.ChatCompletion.create(  
    model="gpt-3.5-turbo",  
    messages=[  
        {"role": "user", "content": q}  
    ]  
)  
  
print(ans['choices'][0]['message']['content'])
```

1. Stocks – Ownership in a company, traded on exchanges.
2. Bonds – Loans to entities, paid back with interest.
3. Mutual Funds – Pooled investments in various assets.
4. ETFs – Similar to mutual funds, but trade like stocks.
5. Real Estate – Ownership or investment in physical property.

Why are AI Applications not yet widely adopted?

The long tail of AI applications will change due to low code and prompt-based development





```
: # Sentiment analysis:
statement1 = "This hotel is dirty"
statement2 = "You are very nice"
instruct = "You are a sentiment analyzer. \
Answer with the user sentiment with one lowercase word"

q = statement2
answer = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": instruct},
        {"role": "user", "content": q}
    ]
)
print(answer.choices[0].message.content)

positive
```

Make AI Think for You

First prompts, questions, responses, different models and response times

We can use ChatGPT from code and get an answer to any question.

This can be: a question from a customer (like an agent), a question for your own work, or a service that is needed for your code... or more.

```
#client = OpenAI()
# This is an example of a chat completion
# We can use it to answer a question

response = myclient.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Who won the last world series?"}
    ]
)
print(response.choices[0].message.content)
```

The Texas Rangers won the 2023 World Series, defeating the Arizona Diamondbacks. This victory marked the first World Series title in the franchise's history.



SCHOOL OF
SCIENCE &
TECHNOLOGY

Make AI Think for You

First prompts, questions, responses, different models and response times

```
# Let's ask another examle question:
question = "What are the most common cars? I want a short list of 5 with 10 words per item"
response = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": question}
    ]
)
print(response.choices[0].message.content)
```

1. Toyota Camry: Reliable, fuel-efficient, spacious, popular choice among consumers, comfortable ride.
2. Honda Civic: Affordable, fuel-efficient, sleek design, reliable reputation, comfortable interior.
3. Ford F-150: Best-selling truck in America, powerful engine options, versatile capabilities.
4. Chevrolet Silverado: Strong towing capacity, rugged durability, variety of trim levels available.
5. BMW 3 Series: Luxury sedan, sporty performance, advanced technology features, upscale interior.

```
# Let's time it so we know how long it takes:
import timeit
from timeit import default_timer as timer
start = timer()

question = "What are the most common cars? I want a short list of 5 with 10 words per item"
response = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": question}
    ]
)

end = timer()
print(response.choices[0].message.content)
print('That took ', end - start, ' seconds')
```

1. Toyota Camry: Reliable, economical, comfortable sedan with a solid reputation.
 2. Honda Civic: Popular compact car known for its fuel efficiency and reliability.
 3. Ford F-150: Best-selling pickup truck with strong performance and versatility.
 4. Chevrolet Silverado: Tough truck with impressive towing capacity and modern features.
 5. Toyota RAV4: Compact SUV known for its spacious interior and reliability.
- That took 1.298694800119847 seconds

```
# Let's try the same question with another model, ie Chat GPT model 4.0:
start = timer()
question = "What are the most common cars? I want a short list of 5 with 10 words per item"
response = myclient.chat.completions.create(
    #model="gpt-4",
    model="gpt-4o",
    messages=[
        {"role": "user", "content": question}
    ]
)
end = timer()
print(response.choices[0].message.content)
print('That took ',end - start,' seconds')
```

1. ****Toyota Corolla**** - Reliable compact car praised for fuel efficiency and durability.
 2. ****Honda Civic**** - Popular compact car renowned for reliability and resale value.
 3. ****Ford F-Series**** - Iconic trucks offering power, versatility, and towing capabilities.
 4. ****Chevrolet Silverado**** - Durable, full-size truck known for strength and dependability.
 5. ****Toyota Camry**** - Midsize sedan with excellent reliability, comfort, and resale value.
- That took 3.2340766999404877 seconds

Turn Prompts Into Products

User input, sentiment analysis, translation, taglines — examples of AI becoming an application feature

```
# Sentiment analysis:
statement1 = "This carpet is never clean"
statement2 = "You are very nice"

q = "what is the sentiment of " + statement1 + ". Answer in one lower case word."
answer = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": q}
    ]
)
print(answer.choices[0].message.content)
```

frustration

```
# Sentiment analysis:
statement1 = "This hotel is dirty"
statement2 = "You are very nice"
instruct = "You are a sentiment analyzer. \
Answer with the user sentiment with one lowercase word which will be positive or negative"

q = statement2
answer = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "system", "content": instruct},
        {"role": "user", "content": q}
    ]
)
print(answer.choices[0].message.content)
```

positive



Turn Prompts Into Products

User input, sentiment analysis, translation, taglines — examples of AI becoming an application feature

```
# easy to translate languages
statement = "I want to buy a new book"
q = "how do you say this in Irish:" + statement

answer = myclient.chat.completions.create(
    model="gpt-3.5-turbo",
    messages=[
        {"role": "user", "content": q}
    ]
)
print(answer.choices[0].message.content)
```

Ba mhaith liom leabhar nua a cheannach.

```
q = "write a tagline for a Ferrari with 15 words or less. Use the word Ferrari"
```

```
answer = myclient.chat.completions.create(  
    model="gpt-3.5-turbo",  
    messages=[  
        {"role": "user", "content": q}  
    ]  
)  
tagline = answer.choices[0].message.content  
print(tagline)
```

Unleash the power of Ferrari and experience the thrill of luxury speed and performance.

Beyond Words: Give AI Eyes & Imagination

Image generation and vision/image understanding

Lets create / generate a picture - inside our code

```
# Use the tagline from the last example. Also can use dall-e-3 model
pic = myclient.images.generate(
    model="dall-e-3",
    prompt=tagline,
    n=1,
    size="1024x1024"
)
image_url = pic.data[0].url
print(image_url)
```



Beyond Words: Give AI Eyes & Imagination

Image generation and vision/image understanding

```
# Example from OpenAI documentation
# https://platform.openai.com/docs/guides/images/usage
pic = myclient.images.generate(
    model="dall-e-3",
    prompt="An IE University typical student",
    n=1,
    size="1024x1024")
image_url = pic.data[0].url
print(image_url)
```



SCHOOL OF
SCIENCE &
TECHNOLOGY

```
# vision - this will enable chatgpt to tell you about a picture
# learn more here: https://platform.openai.com/docs/guides/vision
response = myclient.chat.completions.create(
    model="gpt-4-turbo",
    messages=[
        {
            "role": "user",
            "content": [
                {"type": "text", "text": "What's in this image?"},
                {
                    "type": "image_url",
                    "image_url": {
                        "url": "https://mfalonso.pythonanywhere.com/static/0.jpg"
                    }
                },
            ],
        },
    ],
    max_tokens=300,
)

print(response.choices[0].message)
```



```
import openai
from pathlib import Path

# Define the output file path
speech_file_path = "./speech.mp3"

# Create the speech using OpenAI's latest API structure
response = myclient.audio.speech.create(
    model="tts-1",
    voice="alloy",
    input="Welcome to our Tech Venture Lab February 2025 program in the Tower in Madrid."
)

# Save the generated speech to a file
with open(speech_file_path, "wb") as audio_file:
    audio_file.write(response.content)

print(f"Speech saved to {speech_file_path}")

# Experiment with different voices: alloy, echo, fable, onyx, nova, shimmer
# These voices are optimized for English.
```



```
# speech to text
# this will take that voice file and turn it back to text

#audio_file= open("/path/to/file/audio.mp3", "rb")
audio_file = open("speech.mp3", "rb")
transcript = myclient.audio.transcriptions.create(
    model="whisper-1",
    file=audio_file
)
print(transcript.text)
```

Welcome to our TechVenture Lab February 2025 program in the Tower in Madrid.



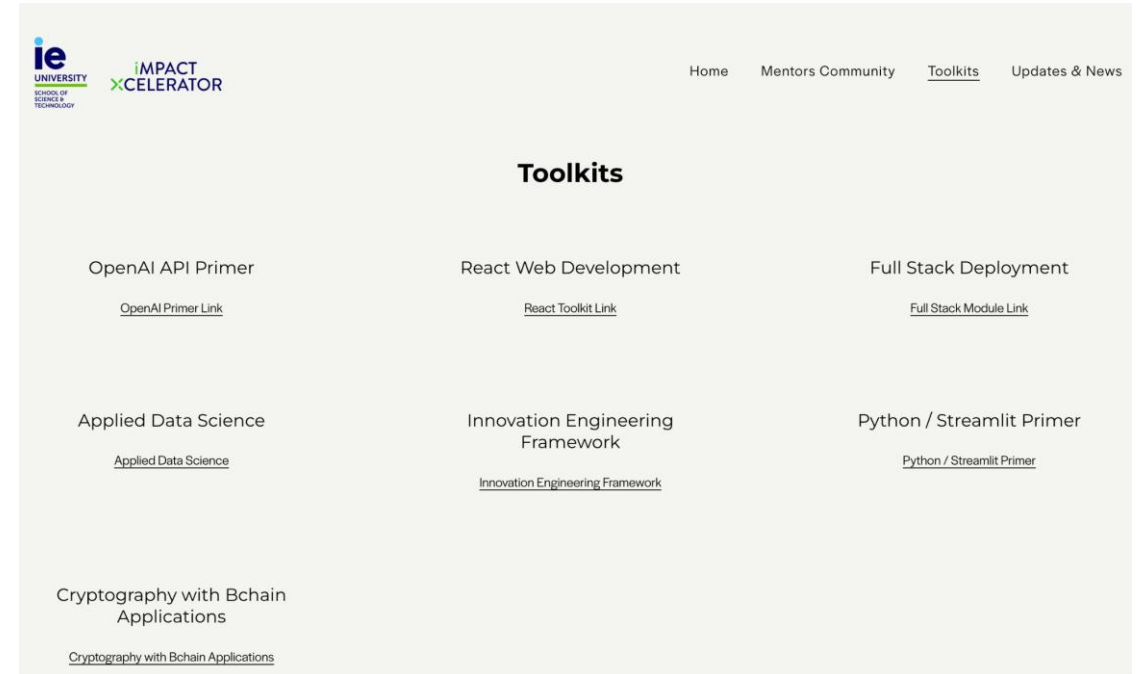
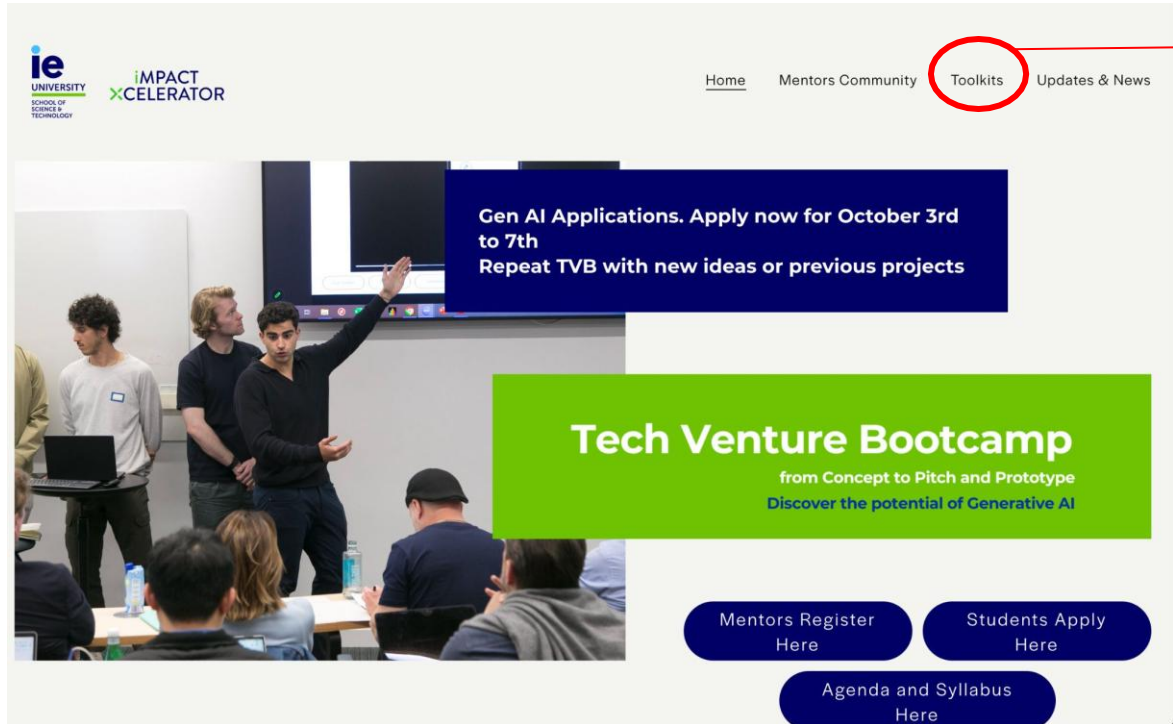
```
# translate language from audio file  
# this will transale a German audio file to English text
```



```
audio_file= open("german.mp3", "rb")  
transcript = myclient.audio.translations.create(  
    model="whisper-1",  
    file=audio_file  
)  
print(transcript.text)
```

The colors. Red, yellow, green, blue, black, white, orange, purple, pink, brown, gray,

<https://www.techventurebootcamp.org/>



platform.openai.com/docs/overview

Keep Log tools & projects TT_Labs networks/partners TickTick My list - Stock Por... ChatGPT Gemini GDrive Dropbox iX.Net Amazon.es

OpenAI Platform Docs API reference Log in Sign up

Search K

GET STARTED

Overview

Quickstart

Models

Changelog

Terms and policies

CAPABILITIES

Text generation

Image generation

Vision

Vector embeddings

Text to speech

Speech to text

Moderation

Reasoning

GUIDES

Function calling

Cookbook

Forum

Help

OpenAI developer platform

Developer quickstart

Set up your environment and make your first API request in minutes

5 min

```
node.js
1 import OpenAI from "openai";
2 const openai = new OpenAI();
3 const completion = await openai.chat.completions.create({
4   model: "gpt-4o",
5   messages: [
6     {"role": "user", "content": "write a haiku about ai"}
7   ]
8 });
```

Meet the models

Pricing Explore all

GPT-4o

Our high-intelligence flagship model for complex, multi-step tasks

Text and image input, text output

128k context length

Smarter model, higher price per token

GPT-4o mini

Our affordable and intelligent small model for fast, lightweight tasks

Text and image input, text output

128k context length

Faster model, lower price per token

o1-preview and o1-mini

Beta

A new series of reasoning models for solving hard problems

Text input, text output

128k context length

Higher latency, uses tokens to think

platform.openai.com

This is where you get sign up for an account and get key.

All the tutorials you will need are here:

Q Search CTRL K

- Get started
 - Overview
 - Quickstart
 - Models
 - Pricing
 - Libraries
- Core concepts
 - Text generation
 - Images and vision
 - Audio and speech
 - Structured output
 - Function calling
 - Using GPT-5
 - Migrate to Responses API
- Tools
 - Using tools
 - Connectors and MCP
 - Web search
 - Code interpreter
 - File search and retrieval

OpenAI developer platform

Developer quickstart

Make your first API request in minutes. Learn the basics of the OpenAI platform.

🕒 5 min

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5",
6   input: "Write a short bedtime story about a unicorn.",
7 });
8
9 console.log(response.output_text);
```

Browse models

[View all](#)



GPT-5
The best model for coding and agentic tasks across domains



GPT-5 mini
A faster, cost-efficient version of GPT-5 for well-defined tasks



GPT-5 nano
Fastest, most cost-efficient version of GPT-5

- Settings
- Your profile
- Organization
- General
- API keys
- Admin keys
- People
- Projects
- Billing**
- Limits
- Usage
- Data controls
- Security
- Project
- General
- API keys
- Webhooks
- People
- Limits

Billing

- Overview**
- Payment methods
- Billing history
- Credit grants
- Preferences

Pay as you go

Credit balance ⓘ

\$4.69

- Add to credit balance
- Cancel plan

Auto recharge is off

ⓘ When your credit balance reaches \$0, your API requests will stop working. Enable automatic recharge to automatically keep your credit balance topped up.

Enable auto recharge

-  **Payment methods**
Add or change payment method
-  **Preferences**
Manage billing information
-  **Pricing**
View pricing and FAQs

-  **Billing history**
View past and future bills
-  **Usage limits**
Set monthly spend limits

Add to credit balance

Amount to add

\$ 5

Enter an amount between \$5 and \$95 [Model pricing](#)

Payment method



+ Add payment method

Cancel Continue

- Create
- Chat
- Audio
- Images
- Assistants
- Manage
 - Usage
 - API keys
 - Logs
 - Storage
 - Batches
- Optimize
- Evaluation
- Fine-tuning

API keys

You have permission to view and manage all API keys in this project.

Do not share your API key with others or expose it in the browser or other client-side code. To protect your account's security, OpenAI may automatically disable any API key that has leaked publicly.

View usage per API key on the [Usage page](#).

NAME	SECRET KEY	CREATED	LAST USED	CREATED BY	PERMISSIONS
TVB	sk-...2TwA	25 feb 2025	30 abr 2025	Manoel Gadi	All

+ Create new secret key

Create new secret key

Owned by

You Service account

This API key is tied to your user and can make requests against the selected project. If you are removed from the organization or project, this key will be disabled.

Name Optional

My Test Key

Project

Default project

Permissions

All Restricted Read only

Cancel

Create secret key

Save your key

Please save your secret key in a safe place since you won't be able to view it again. Keep it secure, as anyone with your API key can make requests on your behalf. If you do lose it, you'll need to generate a new one.

[Learn more about API key best practices](#)

sk-proj-oleSas3JGlgBEz0GMjgRSm4sY8x

Copy

Permissions

Read and write API resources

Done

api_key.txt	25/02/2025 15:26	Documento de texto	1 KB
api_org.txt	25/02/2025 15:30	Documento de texto	1 KB

Search

- Get started
 - Overview
 - Quickstart
 - Models
 - Pricing
 - Libraries
- Core concepts
 - Text generation
 - Images and vision
 - Audio and speech
 - Structured output
 - Function calling
 - Using GPT-5
 - Migrate to Responses API
- Tools
 - Using tools
 - Connectors and MCP
 - Web search
 - Code interpreter
 - File search and retrieval
 - More tools
- Run and scale
 - Conversation state
 - Background mode
 - Streaming

Claim free tokens
Get free tokens on traffic shared with OpenAI.

Pricing

Copy page

Text tokens

Prices per 1M tokens.

Batch Flex Standard Priority

MODEL	INPUT	CACHED INPUT	OUTPUT
gpt-5	\$1.25	\$0.125	\$10.00
gpt-5-mini	\$0.25	\$0.025	\$2.00
gpt-5-nano	\$0.05	\$0.005	\$0.40
gpt-5-chat-latest	\$1.25	\$0.125	\$10.00
gpt-5-codex	\$1.25	\$0.125	\$10.00
gpt-4.1	\$2.00	\$0.50	\$8.00
gpt-4.1-mini	\$0.40	\$0.10	\$1.60
gpt-4.1-nano	\$0.10	\$0.025	\$0.40
gpt-4o	\$2.50	\$1.25	\$10.00
gpt-4o-2024-05-13	\$5.00	-	\$15.00
gpt-4o-mini	\$0.15	\$0.075	\$0.60
gpt-realtime	\$4.00	\$0.40	\$16.00
gpt-4o-realtime-preview	\$5.00	\$2.50	\$20.00
gpt-4o-mini-realtime-preview	\$0.60	\$0.30	\$2.40
gpt-audio	\$2.50	-	\$10.00
gpt-4o-audio-preview	\$2.50	-	\$10.00
gpt-4o-mini-audio-preview	\$0.15	-	\$0.60
o1	\$15.00	\$7.50	\$60.00
o1-pro	\$150.00	-	\$600.00
o3-pro	\$20.00	-	\$80.00
o3	\$2.00	\$0.50	\$8.00
o3-deep-research	\$10.00	\$2.50	\$40.00
o4-mini	\$1.10	\$0.275	\$4.40
o4-mini-deep-research	\$2.00	\$0.50	\$8.00

Text tokens

- Image tokens
- Audio tokens
- Fine tuning
- Built-in tools
- Transcription and speech generation
- Image generation
- Embeddings
- Moderation
- Legacy models

- Create
- Chat
 - Audio
 - Images
 - Assistants
- Manage
- Usage**
 - API keys
 - Logs
 - Storage
 - Batches
- Optimize
- Evaluation
 - Fine-tuning

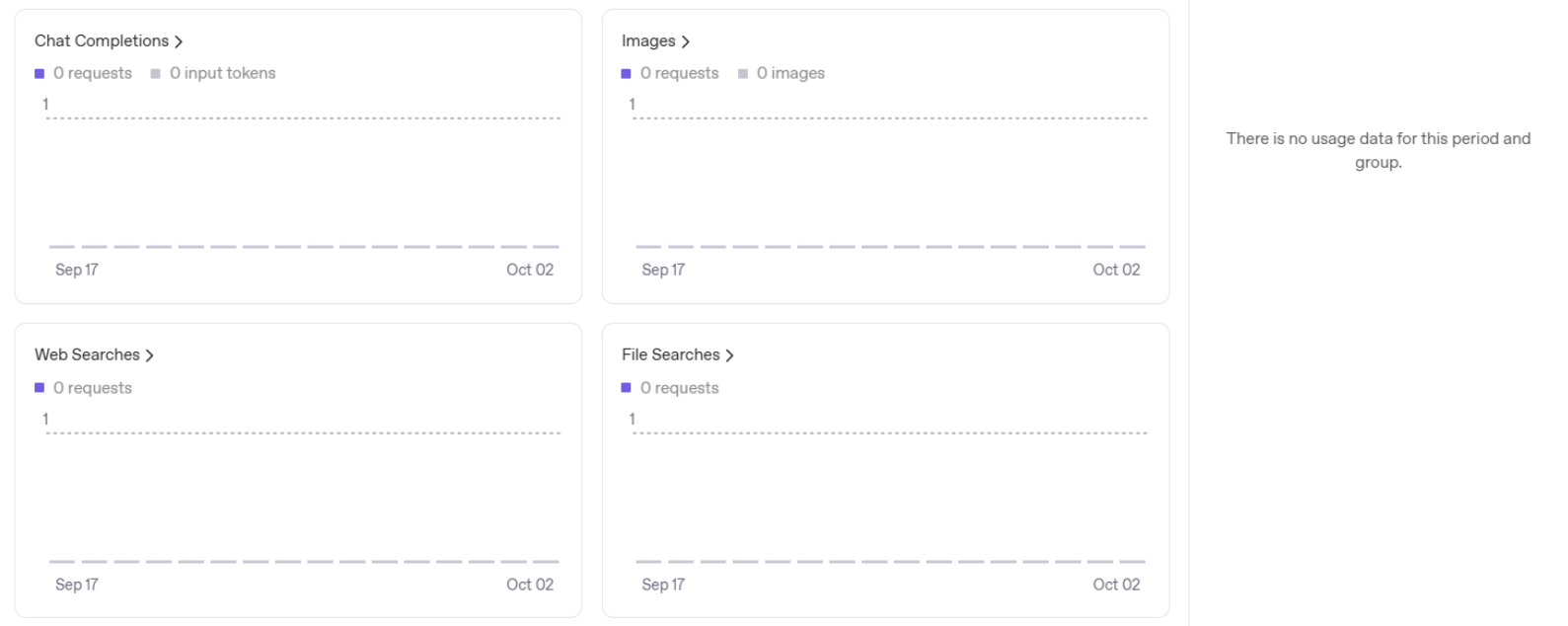
Usage

Default project 09/17/25 - 10/02/25 Export



API capabilities Spend categories

Users Services API Keys



Claim free tokens

Get free tokens on traffic shared with OpenAI.

Plug Into AI

Setup, API keys, authentication, creating the OpenAI client

```
# We will create a client object using the OpenAI api to make requests
# For this we need to use the keys so that it is authenticated and charges can be made
# This is the more recent APY - for 'OpenAI', not 'openai'

import os
from openai import OpenAI
myclient = OpenAI(api_key = api_key, organization=api_key2)
```

TVB 2026 — OpenAI API Lab

Small Flask tests based on the workshop notebook: text, images, vision and audio.

1. Ask AI

Basic prompt → model response.

Question / prompt

What are the most important challenges for an early-stage startup?

Ask

2. Sentiment

The notebook uses a system instruction to classify positive/negative sentiment.

Statement

This product is fantastic and easy to us

Analyze

3. Translation

Change the target language and compare the result.

Text

I want to buy a new book.

Language

Irish

Translate

4. Startup Tagline

The notebook asks AI to generate a short marketing tagline.

Company / startup

Ferrari

Generate tagline

5. Generate an Image

Turn a text prompt into an image.

Image prompt

A cinematic image of a young entrepreneur presenting a new technology startup in Madrid.

Generate image

6. Ask AI About an Image

Paste a publicly accessible image URL. This mirrors the notebook's vision example.

Image URL

https://.../image.jpg

Question

What is in this image? Describe the ma

Analyze image